Estruturas de Dados I

Ponteiros

Professor: Gustavo T. Naozuka
gtnaozuka@uel.br



Universidade
Estadual de Londrina

9 de setembro de 2026

Objetivos da Aula

- Compreender como a memória é **organizada** em endereços.
- Obter o **endereço** de uma variável.
- Declarar **ponteiros** e acessar o valor apontado.
- Relacionar **vetores e ponteiros**.
- Implementar a passagem de parâmetros **por referência** com ponteiros.
- Manipular **registros** por meio de ponteiros.

Memória e Endereços

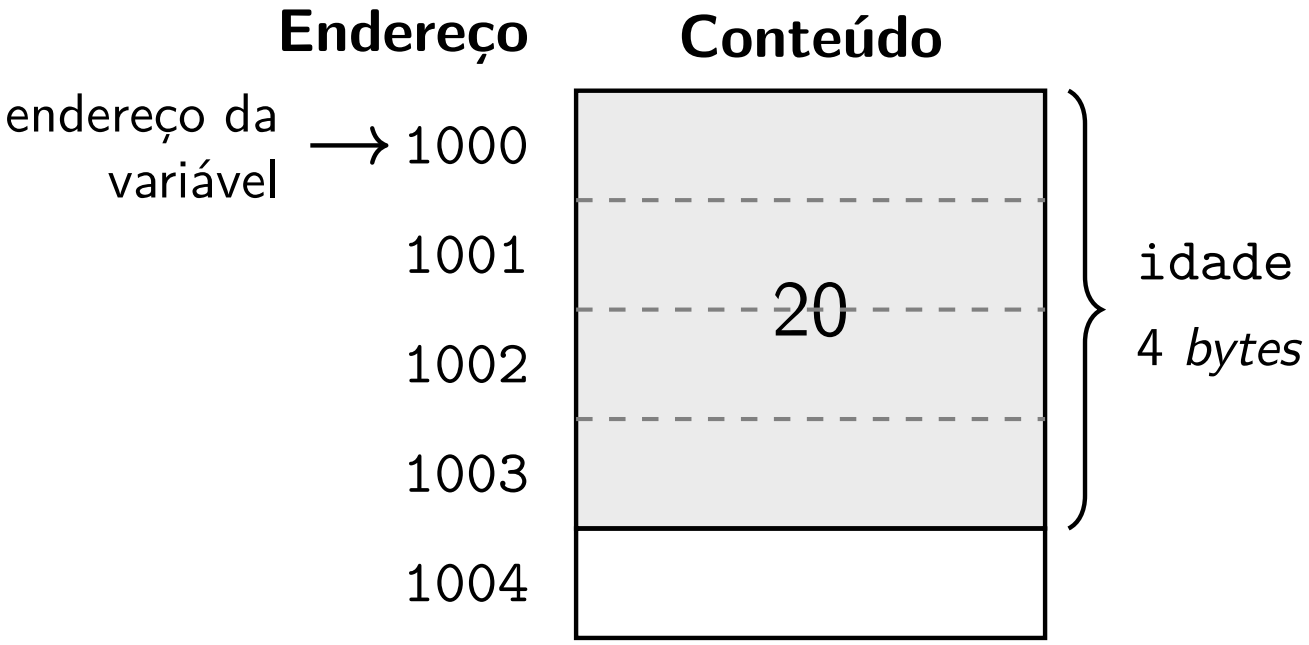
Memória do Computador

- A **memória principal** é uma sequência de células numeradas, cada uma com capacidade para um *byte*.
- O número de uma célula é o seu **endereço**, e é único.
- Declarar uma variável significa **reservar** células consecutivas para armazenar o seu valor.
- O nome da variável é apenas um **apelido**.
- Em C, o endereço é um valor que pode ser obtido, guardado e usado.

Variáveis na Memória

- A declaração `int idade = 20;` reserva 4 células consecutivas:

Figura 1: Ocupação da memória pela variável `idade`.



- O endereço da variável é o da **primeira célula** ocupada.

Operador `sizeof`

- O **operador `sizeof`** informa quantos *bytes* um tipo ou uma variável ocupa:

```c
#include <stdio.h>

int main() {
    printf("char: %zu bytes\n", sizeof(char));
    printf("int: %zu bytes\n", sizeof(int));
    printf("float: %zu bytes\n", sizeof(float));
    printf("double: %zu bytes\n", sizeof(double));

    return 0;
}
```

- O **especificador `%zu`** corresponde ao tipo devolvido por `sizeof`.

Operador &

- O **operador &**, aplicado a uma variável, devolve o seu endereço na memória.

```
int idade = 20;
// idade => Valor armazenado: 20
// &idade => Endereço onde o valor está: 1000
```

- O operador só pode ser aplicado a algo que ocupe memória: expressões como &30 ou &(a + b) não são válidas.
- Toda variável tem um endereço, inclusive vetores e registros.

Imprimindo Endereços

- O **especificador** `%p` imprime um endereço, em notação hexadecimal:

```c
#include <stdio.h>

int main() {
    int a = 10;
    int b = 20;
    double x = 3.5;

    printf("Endereco de a: %p\n", &a);
    printf("Endereco de b: %p\n", &b);
    printf("Endereco de x: %p\n", &x);

    return 0;
}
```

- Os valores impressos mudam a cada execução.

Ponteiros

Ponteiro

- Um endereço é apenas um número, e como todo número pode ser guardado em uma variável.
- Um **ponteiro** é uma variável cujo conteúdo é um endereço de memória.
- Dizemos que um ponteiro **aponta** para uma variável quando contém o endereço dela.
- Todo ponteiro tem um **tipo** associado, que indica o tipo do dado que está no endereço armazenado.

Declaração

- A declaração usa o **operador** * entre o tipo e o nome:

```
int *p;       // Ponteiro para int
double *q;    // Ponteiro para double
char *s;      // Ponteiro para char
```

- Todos os ponteiros armazenam um endereço e ocupam o mesmo espaço, tipicamente 8 *bytes*.
- O operador * pertence à variável, e não ao tipo:

```
int *p, q;    // p = Ponteiro para int; q = int
int *p, *q;   // p, q = Ponteiros para int
```

- Declarar cada ponteiro em uma linha separada evita o problema.

Apontando com Ponteiros

- Um ponteiro recebe **endereços**, obtidos com o operador &:

```
int a = 90;
int *p;

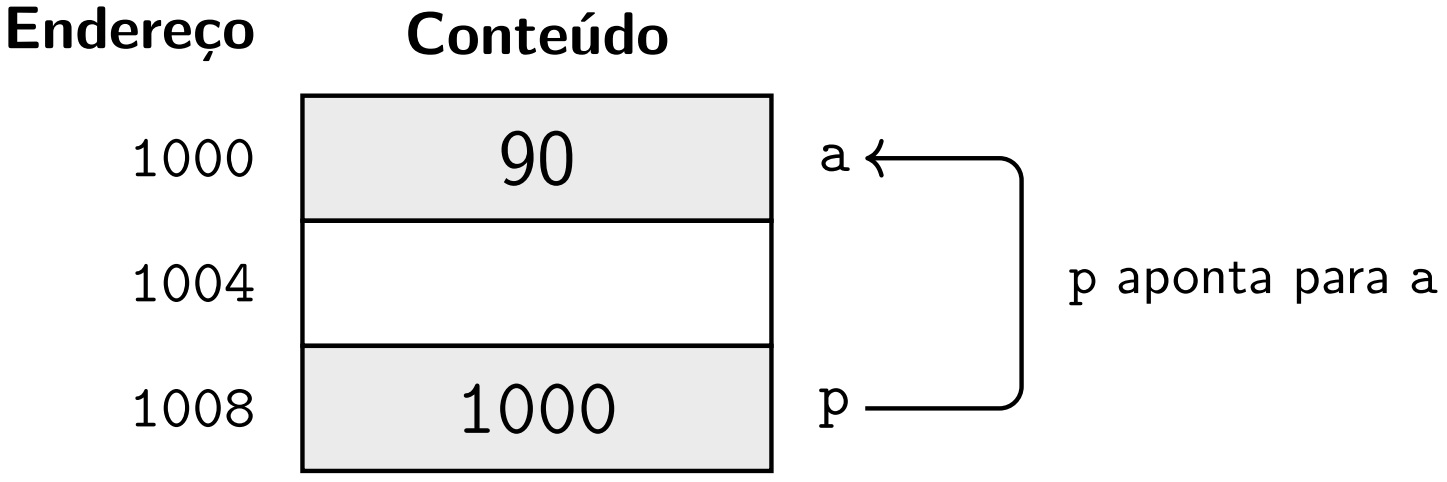
p = &a; // p aponta para a
```

- Os tipos precisam ser **compatíveis**: um `int *` recebe o endereço de um `int`.
- A declaração e a atribuição podem ser reunidas:

```
int a = 90;
int *p = &a;
```

Representação na Memória

Figura 2: Conteúdo da memória após a atribuição `p = &a`.



- O conteúdo de a é o valor 90.
- O conteúdo de p é o endereço 1000, onde a se encontra.

Acessando o Valor Apontado

- O **operador** *, aplicado a um ponteiro, acessa o conteúdo do endereço armazenado.
- Essa operação é chamada de **desreferenciação**.

```c
#include <stdio.h>

int main() {
    int a = 90;
    int *p = &a;

    printf("Endereco de a: %p\n", &a);
    printf("Conteudo de p: %p\n", p);
    printf("Valor de a:    %d\n", a);          // 90
    printf("Valor apontado por p: %d\n", *p);  // 90

    return 0;
}
```

Alterando o Valor Apontado

- A desreferenciação também pode ser utilizada em uma atribuição:

```c
int a = 90;
int *p = &a;

*p = 10; // Altera a variável a

printf("%d\n", a); // 10
```

Endereço de Ponteiros

- Um ponteiro é uma variável como qualquer outra e, portanto, ocupa uma posição de memória:

```c
int a = 90;
int *p = &a;

printf("Endereco de a: %p\n", &a);
printf("Endereco de p: %p\n", &p);
```

- Os endereços impressos são diferentes.
- Um ponteiro pode, inclusive, apontar para outro ponteiro (recurso que será utilizado posteriormente).

Ponteiro Não Inicializado

- Um ponteiro declarado e **não inicializado** contém lixo de memória:

```c
int *p;

*p = 10; // Escreve em um endereço desconhecido
```

- O compilador **aceita** o código, mas o programa escreve em uma posição arbitrária da memória.
- O resultado é **imprevisível**: o programa pode encerrar abruptamente, corromper outra variável ou parecer funcionar.

Valor NULL

- **NULL** é um valor especial que indica que o ponteiro não aponta para lugar algum:

```c
int *p = NULL;

if (p != NULL) {
    printf("%d\n", *p);
} else {
    printf("Ponteiro não aponta para nenhuma variável\n");
}
```

- NULL está definido em `stdlib.h` e também em `stdio.h`.
- Inicializar com NULL e testar antes de desreferenciar é uma prática recomendada.

Endereço de Variável Local

- Uma variável local existe apenas enquanto a função executa. Retornar o seu endereço é um **erro**:

```c
int *cria_valor() {
    int x = 10;
    return &x; // x deixa de existir
}

int main() {
    int *p = cria_valor();
    printf("%d\n", *p); // Conteúdo imprevisível
    return 0;
}
```

- O endereço devolvido continua sendo um número válido, mas a memória foi liberada e pode ser reaproveitada.
- O programa pode imprimir 10, imprimir lixo ou encerrar abruptamente.

Ponteiros e Vetores

Nome do Vetor

- Em C, o **nome de um vetor**, usado sozinho, equivale ao endereço do seu primeiro elemento:

```c
int v[3] = {10, 20, 30};

printf("%p\n", v);       // Endereço do primeiro elemento
printf("%p\n", &v[0]);   // Mesmo endereco
```

- Um ponteiro pode, portanto, receber o vetor **diretamente**:

```c
int *p = v; // Equivale a int *p = &v[0];
```

Aritmética de Ponteiros

- Somar um número inteiro a um ponteiro produz um novo endereço:

```c
int v[3] = {10, 20, 30};
int *p = v;

printf("%p\n", p);       // Endereço de v[0]
printf("%p\n", p + 1);   // Endereço de v[1]
printf("%p\n", p + 2);   // Endereço de v[2]
```

- A expressão `p + 1` avança para o **próximo elemento**.
- O deslocamento em *bytes* é calculado pelo compilador a partir do **tipo do ponteiro**.

Equivalência entre Colchetes e Asterisco

- Se `p + i` é o endereço do elemento `i`, então `*(p + i)` é o próprio elemento:

```c
int v[3] = {10, 20, 30};
int *p = v;

printf("%d\n", v[1]);       // 20
printf("%d\n", *(p + 1));   // 20
printf("%d\n", *(v + 1));   // 20
printf("%d\n", p[1]);       // 20
```

Percorrendo um Vetor com Ponteiro

- Um vetor também pode ser **percorrido** movendo o próprio ponteiro:

```c
#include <stdio.h>

int main() {
    int v[5] = {10, 20, 30, 40, 50};
    int *p;

    for (p = v; p < v + 5; p++) {
        printf("%d\n", *p);
    }

    return 0;
}
```

- A condição `p < v + 5` compara endereços.

Percorrendo Cadeias de Caracteres com Ponteiro

- Uma cadeia é um vetor de `char` e, portanto, também pode ser **percorrida** por um ponteiro:

```c
#include <stdio.h>

int main() {
    char nome[] = "Estruturas";
    char *p;

    for (p = nome; *p != '\0'; p++) {
        printf("%c\n", *p);
    }

    return 0;
}
```

Vetor de Caracteres e Ponteiro para Literal

- As duas declarações a seguir se parecem, mas produzem resultados **distintos**:

```
char a[] = "Dados"; // Vetor com uma cópia da cadeia
char *b = "Dados";  // Ponteiro para uma cadeia constante

a[0] = 'L';   // Permitido
b[0] = 'L';   // Encerra o programa
```

- No primeiro caso, a cadeia é copiada para um vetor e pode ser alterada.
- No segundo caso, o ponteiro aponta para uma região somente de leitura do programa.
- Cadeias que serão modificadas devem ser declaradas como vetor.

Comparando e Subtraindo Ponteiros

- Dois ponteiros para o mesmo vetor podem ser **comparados e subtraídos**:

```c
int v[5] = {10, 20, 30, 40, 50};
int *inicio = &v[0];
int *fim = &v[4];

int distancia = fim - inicio;

printf("%d\n", distancia);          // 4
printf("%d\n", inicio < fim);       // 1
```

- A subtração devolve a **quantidade de elementos** entre os dois endereços.

Diferenças entre Vetor e Ponteiro

- Apesar da equivalência de uso, existem **diferenças** entre vetor e ponteiro.

Tabela 1: Comparação entre um vetor e um ponteiro para o seu primeiro elemento.

	Vetor `int v[5]`	**Ponteiro** `int *p`
Memória reservada	5 elementos	Apenas um endereço
`sizeof`	20 *bytes*	8 *bytes*
Pode ser reatribuído	Não	Sim
Pode ser modificado com aritmética	Não	Sim

- A atribuição `v = p` é rejeitada pelo compilador, já `p = v` é válida.
- O incremento `v++` é rejeitado pelo compilador, já `p++` é válido.

Passagem de Parâmetros por Referência

Problema em Aberto

- **Problema:** trocar os valores de duas variáveis.

```c
void troca(int x, int y) {
    int aux = x;
    x = y;
    y = aux;
}

int main() {
    int a = 10, b = 20;
    troca(a, b);
    printf("%d %d\n", a, b); // 10 20
    return 0;
}
```

- A função recebe **cópias** de `a` e `b`, troca as cópias e as descarta ao terminar.

Passagem por Referência

- Na **passagem por referência**, o valor passado é um endereço.
- O parâmetro é declarado como **ponteiro** e a chamada usa o **operador &**:

```c
void incrementa(int *x) {
    *x = *x + 1; // Altera a variável apontada
}

int main() {
    int a = 1;
    incrementa(&a);
    printf("%d\n", a); // 2
    return 0;
}
```

Solução do Problema em Aberto

- Com ponteiros, a função `troca()` altera as variáveis do `main()`:

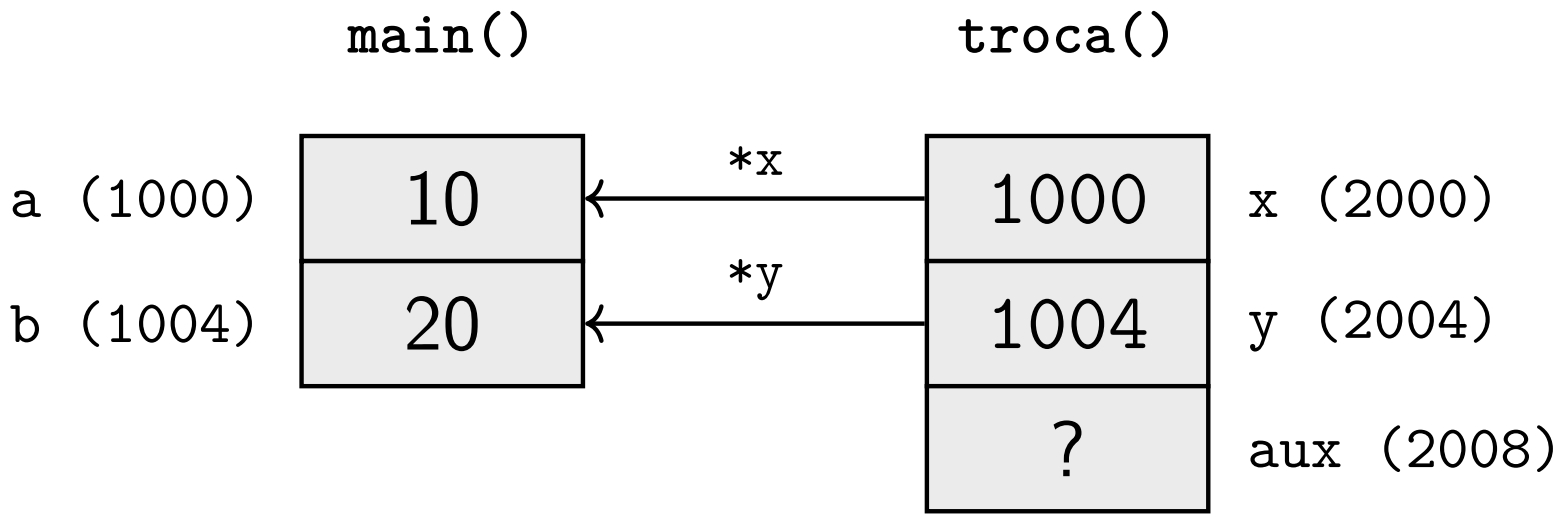
```c
void troca(int *x, int *y) {
    int aux = *x;
    *x = *y;
    *y = aux;
}

int main() {
    int a = 10, b = 20;
    troca(&a, &b);
    printf("%d %d\n", a, b); // 20 10
    return 0;
}
```

Representação na Memória

```
void troca(int *x, int *y) { // <-- Chamada: troca(&a, &b)
    int aux = *x;
    *x = *y;
    *y = aux;
}
```

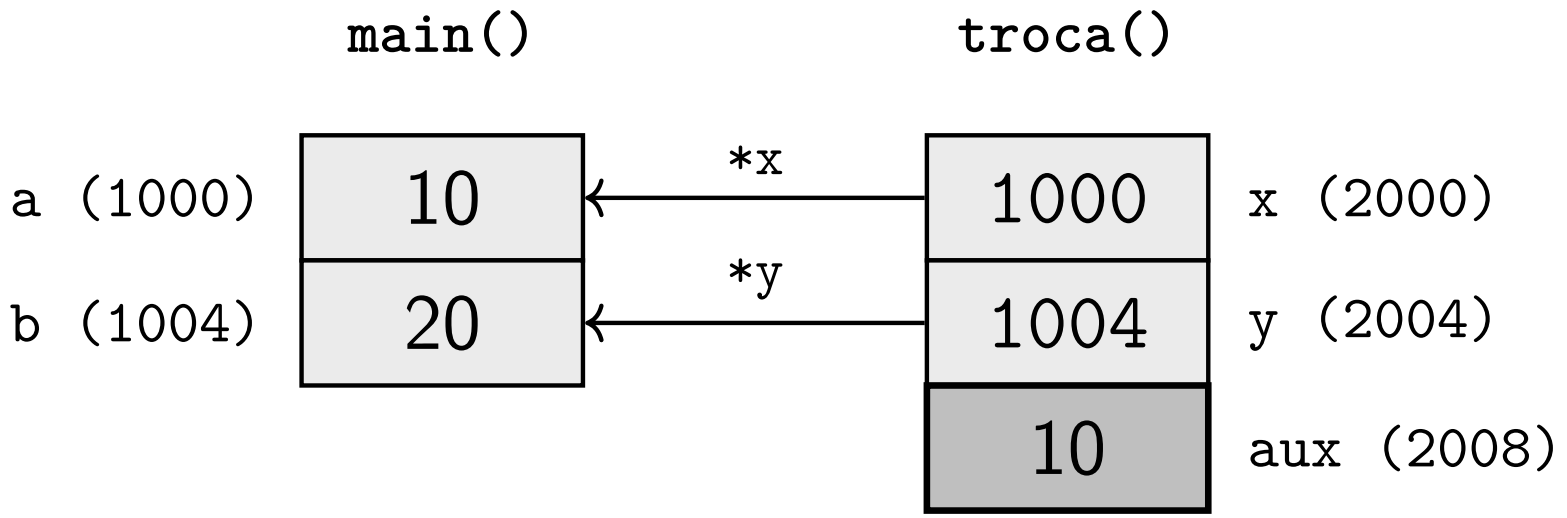
Figura 3: Estado da memória ao iniciar a execução da função.



Representação na Memória

```
void troca(int *x, int *y) {
    int aux = *x; // <-- Executando
    *x = *y;
    *y = aux;
}
```

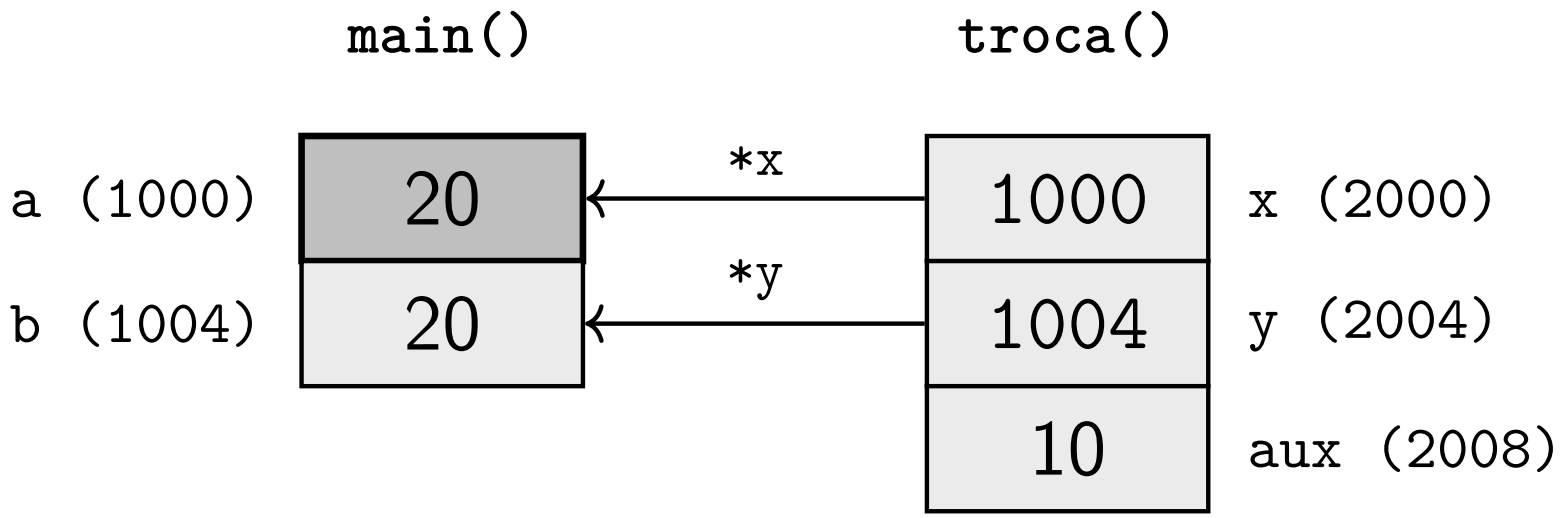
Figura 4: A variável aux recebe o valor apontado por x.



Representação na Memória

```
void troca(int *x, int *y) {
    int aux = *x;
    *x = *y; // <-- Executando
    *y = aux;
}
```

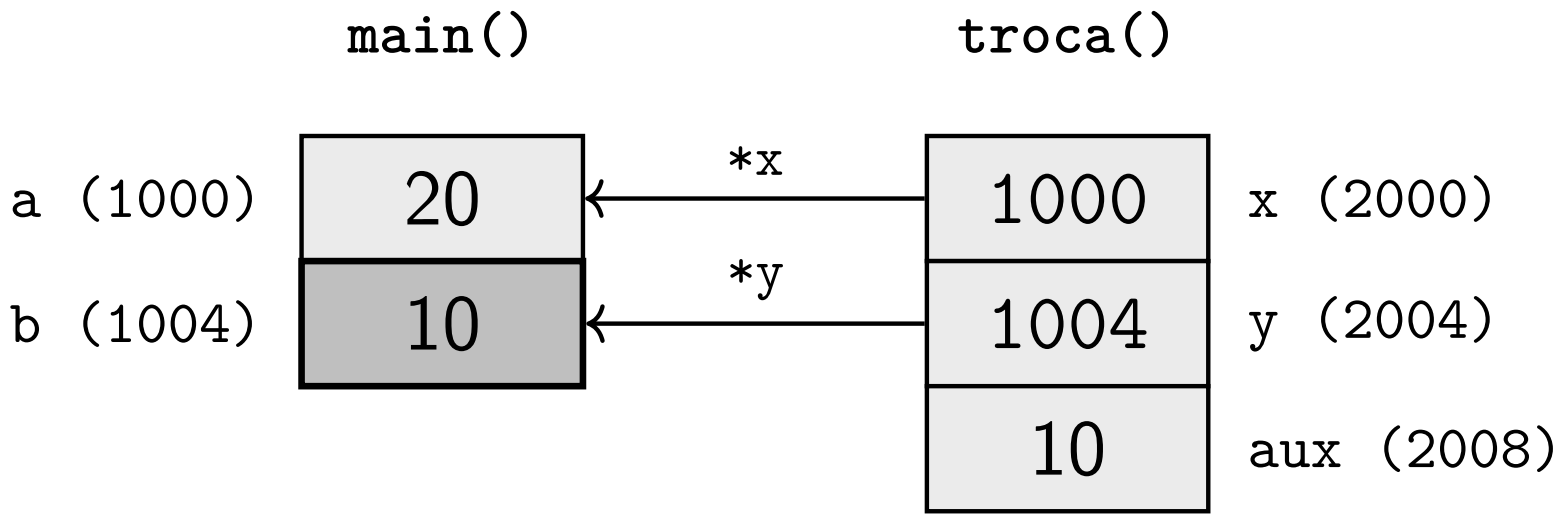
Figura 5: O valor apontado por y é escrito no endereço apontado por x.



Representação na Memória

```
void troca(int *x, int *y) {
    int aux = *x;
    *x = *y;
    *y = aux; // <-- Executando
}
```

Figura 6: O valor guardado em aux é escrito no endereço apontado por y.



Retornando Múltiplos Valores

- O return produz um único valor. Com ponteiros, uma função pode devolver **vários valores**:

```c
void divide(int a, int b, int *quociente, int *resto) {
    *quociente = a / b;
    *resto = a % b;
}

int main() {
    int q, r;
    divide(17, 5, &q, &r);
    printf("Quociente: %d, Resto: %d\n", q, r);
    // Quociente: 3, Resto: 2
    return 0;
}
```

- Parâmetros usados dessa forma são chamados de **parâmetros de saída**.

Retornando Múltiplos Valores

- Uma combinação comum reserva o `return` para indicar se a operação foi um **sucesso ou falha**:

```
int tenta_dividir(int a, int b, double *resultado) {
    if (b == 0)
        return 0;   // Falha
    *resultado = (double) a / b;
    return 1;       // Sucesso
}

int main() {
    double r;
    if (tenta_dividir(10, 0, &r))
        printf("%lf\n", r);
    else
        printf("Divisao por zero.\n");
    return 0;
}
```

Protegendo o Parâmetro

- Receber um endereço permite alterar o original, ainda que a função não pretenda fazê-lo.
- O **qualificador `const`** declara que a função não altera os dados recebidos:

```
void imprime(const int *v, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d\n", v[i]);
    }
}
```

- Uma alteração no vetor `v` dentro dessa função é **recusada** na compilação.

Registros: Problema em Aberto

- **Problema:** a passagem de um registro também atuou sobre uma cópia.

```c
void aprovar(Aluno a) {
    a.nota1 = 10.0; // Altera apenas a cópia
    a.nota2 = 10.0;
}

int main() {
    Aluno x = {"Ana Souza", 20260145, 8.5, 7.0};

    aprovar(x);
    printf("%.2f %.2f\n", x.nota1, x.nota2); // 8.50 7.00

    return 0;
}
```

Ponteiro para Registro

- Um **ponteiro para registro** é declarado e inicializado como qualquer outro:

```
Aluno x = {"Ana Souza", 20260145, 8.5, 7.0};
Aluno *p = &x;
```

- O acesso a um campo exige **desreferenciar** o ponteiro antes de aplicar o ponto:

```
printf("%.2f %.2f\n", (*p).nota1, (*p).nota2);
// 8.50 7.00
```

- Os parênteses são **obrigatórios**.
- C oferece uma abreviação para a construção `(*p).campo` utilizando o **operador** `->`:

```
printf("%.2f %.2f\n", p->nota1, p->nota2); // 8.50 7.00
```

Registros: Solução do Problema em Aberto

- Com o endereço do registro, a função altera o original:

```c
void aprovar(Aluno *a) {
    a->nota1 = 10.0;
    a->nota2 = 10.0;
}

int main() {
    Aluno x = {"Ana Souza", 20260145, 8.5, 7.0};
    aprovar(&x);
    printf("%.2f %.2f\n", x.nota1, x.nota2);
    // 10.00 10.00
    return 0;
}
```

Registros: Custo da Cópia

- Passar um registro por valor copia **todos os seus campos**.
- Um ponteiro ocupa sempre **8 *bytes***, qualquer que seja o tamanho do registro apontado.
- Por isso registros costumam ser passados **por referência** mesmo quando não são alterados.

Estruturas de Dados I

Alocação Dinâmica de Memória

Professor: Gustavo T. Naozuka
gtnaozuka@uel.br



Universidade
Estadual de Londrina

16 de setembro de 2026

Objetivos da Aula

- Distinguir o armazenamento **estático** do armazenamento **dinâmico**.
- Reconhecer as **regiões de memória** de um programa em execução.
- **Solicitar** memória durante a execução, **redimensionar** uma área já alocada e **devolver** a memória ao sistema.
- Construir vetores, matrizes e registros **dinamicamente**.

Armazenamento Estático × Dinâmico

Vetor de Tamanho Fixo

- A declaração de um vetor **reserva** um espaço definido antes da execução:

```
#define MAX 1000

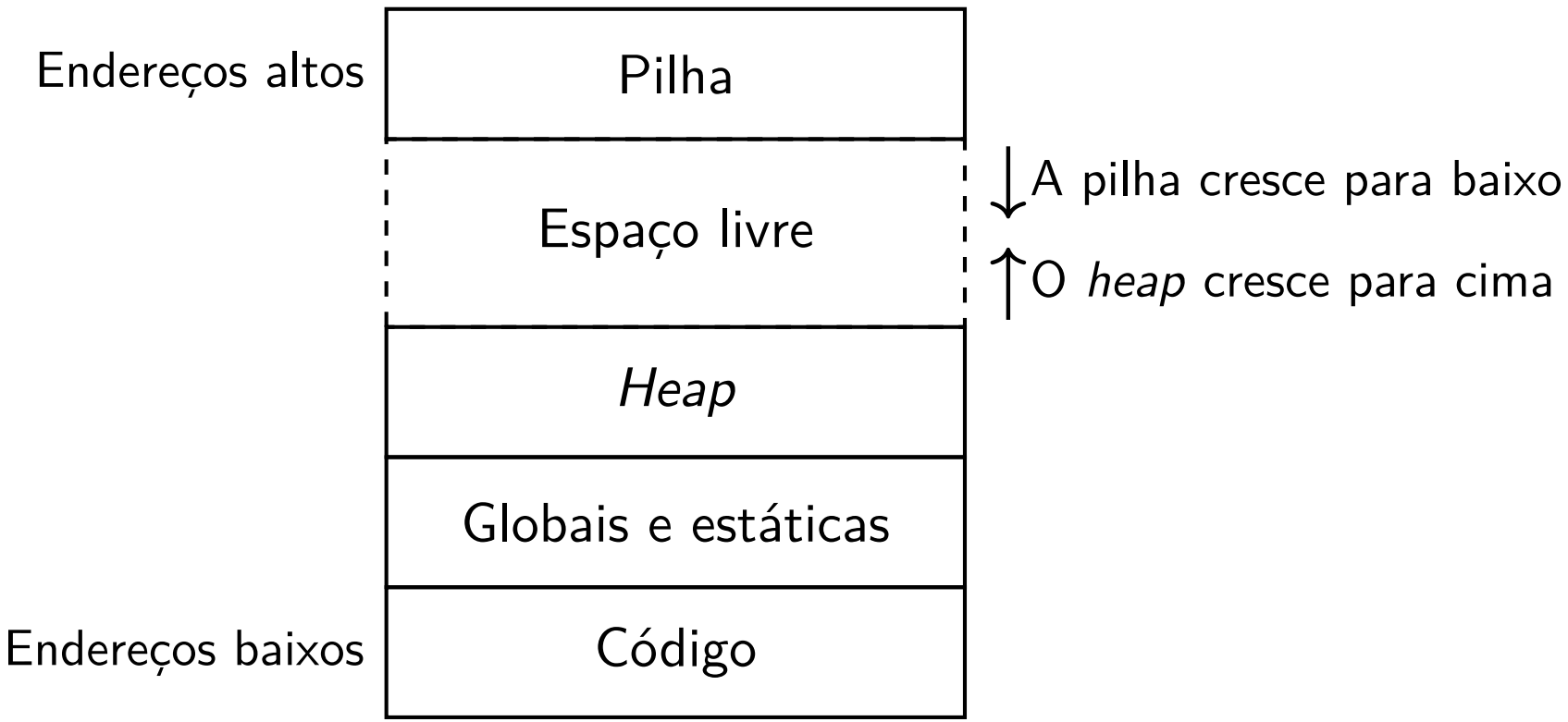
int v[MAX];
```

- O tamanho está escrito no código e **não muda** enquanto o programa executa.
- Devemos declarar um vetor com **capacidade suficiente** e utilizar apenas parte dele.
- Isso desperdiça memória na maior parte das execuções e ainda **limita** o programa, que não trata entradas maiores que `MAX`.

Regiões de Memória de um Programa

- A memória de um programa em execução é dividida em **regiões** com finalidades distintas.

Figura 1: Organização da memória de um programa em execução.



Regiões de Memória: Código

- A região de **código** guarda as instruções do programa, geradas pelo compilador.
- É carregada no início da execução e permanece inalterada até o fim.
- O tamanho é definido na compilação e não depende dos dados processados.

Regiões de Memória: Variáveis Globais e Estáticas

- Nessa região ficam as variáveis declaradas **fora** de qualquer função:

```
int contador = 0;  // Variável global

void incrementa() {
    contador++;
}
```

- O espaço é reservado **antes** do início da execução e só é liberado no encerramento do programa.
- O tamanho é fixo, definido na compilação.

Regiões de Memória: Pilha

- Os **parâmetros** e as **variáveis locais** de uma função ficam na pilha.
- O espaço é criado na chamada da função e destruído no retorno:

```
int soma(int a, int b) {
    int r = a + b;  // a, b, r ocupam a pilha
    return r;       // O espaço é liberado no retorno
}
```

- A reserva e a liberação são automáticas, feitas pelo compilador.

Erro: Endereço de Variável Local

- Vimos que uma variável local deixa de existir no retorno da função. O mesmo vale para um vetor:

```
int *cria_vetor() {
    int v[5] = {10, 20, 30, 40, 50};
    return v; // v deixa de existir
}
```

- O compilador apenas alerta, mas gera o executável:

```
// warning: function returns address of local variable
```

- Não conseguimos, portanto, construir um vetor dentro de uma função e devolvê-lo para quem a chamou.
- Precisamos de uma região de memória que **não** seja liberada no retorno da função.

Regiões de Memória: *Heap*

- O ***heap*** (também chamado de memória livre ou monte) é a região reservada para atender pedidos feitos durante a execução.
- O programa solicita um bloco de um tamanho qualquer e recebe o endereço inicial dele.
- O tamanho do pedido pode ser calculado durante a execução, a partir de um dado lido do usuário ou de um arquivo.
- O bloco permanece válido até ser liberado explicitamente, mesmo após o retorno da função que o alocou.
- Todo acesso ao bloco é feito por meio de um ponteiro.

Armazenamento Estático × Dinâmico

Tabela 1: Armazenamento estático × dinâmico.

Característica	**Estático (pilha)**	**Dinâmico (*heap*)**
Momento da reserva	Antes da execução	Durante a execução
Tamanho	Fixo no código	Calculado em execução
Redimensionamento	Impossível	Possível
Tempo de vida	Até o fim da função	Até a liberação explícita
Liberação	Automática	Responsabilidade do programador
Forma de acesso	Nome da variável	Ponteiro

Alocação com `malloc`

Pedido de Memória

- O **pedido de memória** é feito em *bytes* utilizando o operador `sizeof`.
- O tipo devolvido por `sizeof` é `size_t`, um inteiro sem sinal usado para representar tamanhos.
- O espaço necessário para `n` elementos é `n * sizeof(tipo)`.

Função `malloc()`

- A **função** `malloc()`, declarada em `stdlib.h`, solicita um bloco de memória no *heap*:

```
void *malloc(size_t tamanho);
```

- O **parâmetro** é o tamanho do bloco em *bytes*.
- O **retorno** é o endereço do primeiro *byte* do bloco reservado. Se não houver memória disponível, o retorno é `NULL`.
- Esse endereço precisa ser guardado em um **ponteiro**.
- O bloco reservado é **contíguo**.

Alocando um Único Valor

- O espaço para um inteiro é reservado e acessado por meio de um ponteiro:

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p;
    p = (int *) malloc(sizeof(int));
    *p = 90;
    printf("Endereço: %p\n", p);   // Endereço no heap
    printf("Conteúdo: %d\n", *p);  // 90
    return 0;
}
```

Retorno `void *` e Conversão de Tipo

- A função `malloc()` apenas reserva *bytes* e não sabe o que será guardado neles.
- Por isso o retorno é do **tipo** `void *`, um ponteiro sem tipo associado.
- A **conversão** indica ao compilador como interpretar o conteúdo do bloco:

```
int *p = (int *) malloc(sizeof(int));
double *q = (double *) malloc(sizeof(double));
```

- O tipo do ponteiro também determina o deslocamento usado na aritmética de ponteiros.

Verificando o Sucesso da Alocação

- O pedido de memória pode ser recusado, e o retorno nesse caso é NULL.
- Devemos verificar o sucesso da alocação antes de qualquer uso do ponteiro:

```
int *p = (int *) malloc(sizeof(int));

if (p == NULL) {
    printf("Memória insuficiente\n");
    return 1;
}

*p = 90;
```

Alocando um Vetor

- Um vetor de `n` elementos exige `n * sizeof(tipo)` *bytes*:

```
int *v = (int *) malloc(10 * sizeof(int));
```

- O ponteiro passa a ser usado exatamente como um vetor:

```
for (int i = 0; i < 10; i++) {
    v[i] = i * 10;
}
```

- A diferença está na **origem** do espaço: ele foi pedido em execução, e não reservado pelo compilador.

Liberação com `free`

Função free()

- A **função free()**, declarada em stdlib.h, devolve ao sistema um bloco obtido por alocação dinâmica:

```
void free(void *ponteiro);
```

- O **argumento** precisa ser exatamente o endereço devolvido pela alocação:

```
int *v = (int *) malloc(10 * sizeof(int));
free(v);
```

- Uma única chamada libera o bloco inteiro, e não um elemento por vez.
- Cada alocação bem-sucedida **deve** ter uma liberação correspondente.

Vazamento de Memória

- Um bloco não liberado permanece ocupado até o encerramento do programa.
- Perder o endereço torna o bloco inalcançável:

```
int *v = (int *) malloc(10 * sizeof(int));
v = (int *) malloc(20 * sizeof(int));
// O endereço do bloco dos 10 inteiros foi perdido
```

- Essa situação é chamada de **vazamento de memória**.
- O efeito é pequeno em um programa curto, mas se acumula em laços e em programas de execução prolongada, até esgotar a memória disponível.

Ponteiro Pendente

- A função `free()` devolve o bloco, mas **não altera** o ponteiro:

```
int *p = (int *) malloc(sizeof(int));
*p = 90;
free(p);
printf("%d\n", *p); // Acesso àmemória já liberada
```

- O ponteiro continua guardando um endereço que não pertence mais ao programa.
- A prática recomendada é **anular** o ponteiro logo após a liberação:

```
free(p);
p = NULL;
```

Liberações Indevidas

- Liberar duas vezes o mesmo bloco encerra o programa:

```
free(p);
free(p); // double free or corruption
```

- Liberar um endereço deslocado também é inválido:

```
int *v = (int *) malloc(10 * sizeof(int));
v++;          // v aponta para o segundo elemento
free(v);      // free(): invalid pointer
```

Ciclo de Vida de um Bloco Dinâmico

- As cinco etapas a seguir se repetem em qualquer uso de memória dinâmica:

```
int *v;

v = (int *) malloc(n * sizeof(int));  // 1. Alocar
if (v == NULL) {                      // 2. Testar
    return 1;
}
for (i = 0; i < n; i++) {             // 3. Usar
    v[i] = 0;
}
free(v);                              // 4. Liberar
v = NULL;                             // 5. Anular
```

Alocando e Liberando um Vetor

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, *v;
    printf("Quantos valores? ");
    scanf("%d", &n);
    v = (int *) malloc(n * sizeof(int));
    if (v == NULL) {
        printf("Memória insuficiente\n");
        return 1;
    }
    for (int i = 0; i < n; i++)
        scanf("%d", &v[i]);
    free(v);
    return 0;
}
```

Alocação com `calloc`

Função calloc()

- A **função** `calloc()` aloca espaço para um conjunto de elementos de mesmo tipo:

```
void *calloc(size_t quantidade, size_t tamanho);
```

- O primeiro parâmetro é a quantidade de elementos e o segundo é o tamanho de cada um.
- O espaço reservado é o produto dos dois valores:

```
int *v = (int *) calloc(10, sizeof(int));
```

- Todos os *bytes* do bloco são preenchidos com **zero**.
- O retorno e a verificação de `NULL` seguem as mesmas regras da função `malloc()`.

Alocando e Inicializando um Vetor

- O vetor pode ser lido logo após a alocação:

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *v;
    v = (int *) calloc(5, sizeof(int));
    if (v == NULL) {
        printf("Memória insuficiente\n");
        return 1;
    }
    for (int i = 0; i < 5; i++)
        printf("%d ", v[i]); // 0 0 0 0 0
    free(v);
    return 0;
}
```

Redimensionamento com `realloc`

Tamanho Variável Durante a Execução

- Nem sempre a quantidade de dados é conhecida no momento da alocação.
- Um programa que lê valores até o usuário encerrar a entrada não sabe de antemão quantos serão.
- A **solução** é começar com um bloco pequeno e aumentá-lo conforme a necessidade.
- Isso exige uma operação capaz de **mudar o tamanho** de um bloco já alocado, preservando o que ele contém.

Função realloc()

- A **função `realloc()`** ajusta o tamanho de um bloco existente:

```
void *realloc(void *ponteiro, size_t novo_tamanho);
```

- O primeiro parâmetro é o endereço devolvido por uma alocação anterior, e o segundo é o novo tamanho do bloco, em *bytes*.
- O conteúdo existente é **preservado** até o limite do menor dos dois tamanhos.
- Em um aumento, o espaço acrescentado **não** é inicializado.
- O retorno é o endereço do bloco redimensionado.

Função realloc()

- Se não houver espaço contíguo suficiente após o bloco atual, o sistema:
  - Reserva um novo bloco em outra região;
  - Copia o conteúdo antigo para o novo bloco;
  - Libera o bloco original.
- Por isso o endereço precisa ser sempre reatribuído:

```
int *v = (int *) malloc(5 * sizeof(int));
v = (int *) realloc(v, 10 * sizeof(int));
```

Uso de um Ponteiro Auxiliar

- Quando não há memória disponível, a função `realloc()` devolve `NULL` e mantém o bloco original intacto.
- Atribuir o retorno diretamente ao ponteiro original perde esse bloco:

```
v = (int *) realloc(v, n * sizeof(int));
// Se o retorno for NULL, o bloco antigo foi perdido
```

- A forma segura guarda o retorno em um **ponteiro auxiliar**:

```
int *aux = (int *) realloc(v, n * sizeof(int));
if (aux == NULL) {
    free(v); // O bloco antigo precisa ser liberado
    return 1;
}
v = aux;
```

Redimensionando um Vetor

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 0, capacidade = 2, valor;
    int *v = (int *) malloc(capacidade * sizeof(int));

    scanf("%d", &valor);
    while (valor >= 0) {
        if (n == capacidade) {
            capacidade = capacidade * 2;
            v = (int *) realloc(v,
                capacidade * sizeof(int));
        }
        ...
```

Redimensionando um Vetor

```
        ...
        v[n] = valor;
        n++;
        scanf("%d", &valor);
    }
    printf("%d valores lidos\n", n);
    free(v);
    return 0;
}
```

Alocação Dinâmica de Registros

Alocando um Registro

- O procedimento é o mesmo usado com os tipos básicos:

```
typedef struct {
    int matricula;
    float nota1;
    float nota2;
} Aluno;

Aluno *p = (Aluno *) malloc(sizeof(Aluno));
```

- O operador `sizeof` calcula o espaço do registro inteiro, somando todos os campos.
- O bloco reservado é **contíguo** e comporta todos os campos.

Acessando os Campos e Liberando o Registro

- O acesso é feito com o operador `->`, como em qualquer ponteiro para registro:

```
Aluno *p = (Aluno *) malloc(sizeof(Aluno));
if (p == NULL)
    return 1;

p->matricula = 20260145;
p->nota1 = 8.5;
p->nota2 = 7.0;
printf("%d %.2f\n", p->matricula,
    (p->nota1 + p->nota2) / 2);

free(p);
```

- Uma única chamada a `free()` devolve o registro completo.

Vetor Dinâmico de Registros

- Para criar um **vetor dinâmico de registros**, multiplica-se a quantidade de registros pelo tamanho de um deles:

```
Aluno *turma = (Aluno *) malloc(n * sizeof(Aluno));
```

- Cada elemento é um registro, acessado pelo índice e pelo ponto:

```
turma[0].matricula = 20260145;
turma[0].nota1 = 9.0;
turma[0].nota2 = 8.0;
```

Exemplo Completo

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int matricula;
    float nota1;
    float nota2;
} Aluno;

int main() {
    int n;
    Aluno *turma;

    printf("Quantos alunos? ");
    scanf("%d", &n);
    ...
```

Exemplo Completo

```
    ...
    turma = (Aluno *) malloc(n * sizeof(Aluno));
    if (turma == NULL)
        return 1;

    for (int i = 0; i < n; i++) {
        scanf("%d %f %f", &turma[i].matricula,
              &turma[i].nota1, &turma[i].nota2);
    }

    for (int i = 0; i < n; i++) {
        printf("%d %.2f\n", turma[i].matricula,
               (turma[i].nota1 + turma[i].nota2) / 2);
    }
    free(turma);
    return 0;
}
```

Alocação Dinâmica de Matrizes

Matriz de Tamanho Fixo

- Uma matriz declarada tem as duas dimensões **fixadas** no código:

```
int m[3][4];
```

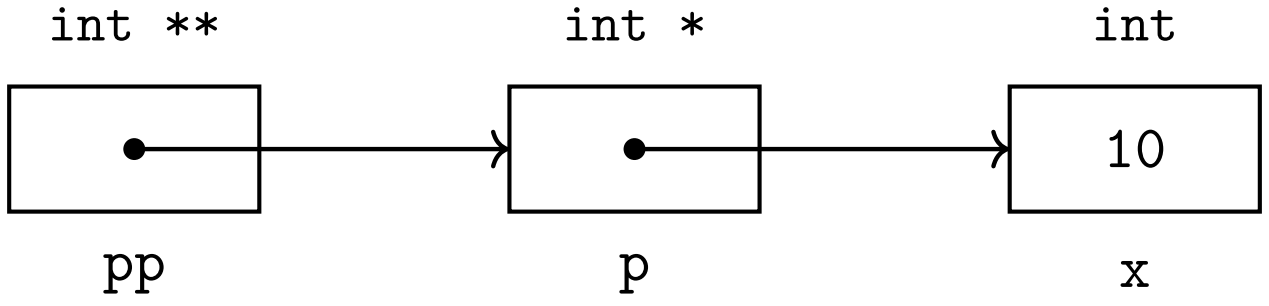
- O problema é o mesmo do vetor, agora em duas dimensões.
- Queremos que o número de linhas e de colunas seja lido **durante** a execução.
- A solução usa um recurso novo: um **ponteiro para ponteiro**.

Ponteiro para Ponteiro

- Um ponteiro é uma variável e, como qualquer variável, tem **endereço**.
- Esse endereço pode ser guardado em **outro ponteiro**:

```
int x = 10;
int *p = &x;     // p guarda o endereço de x
int **pp = &p;   // pp guarda o endereço de p
```

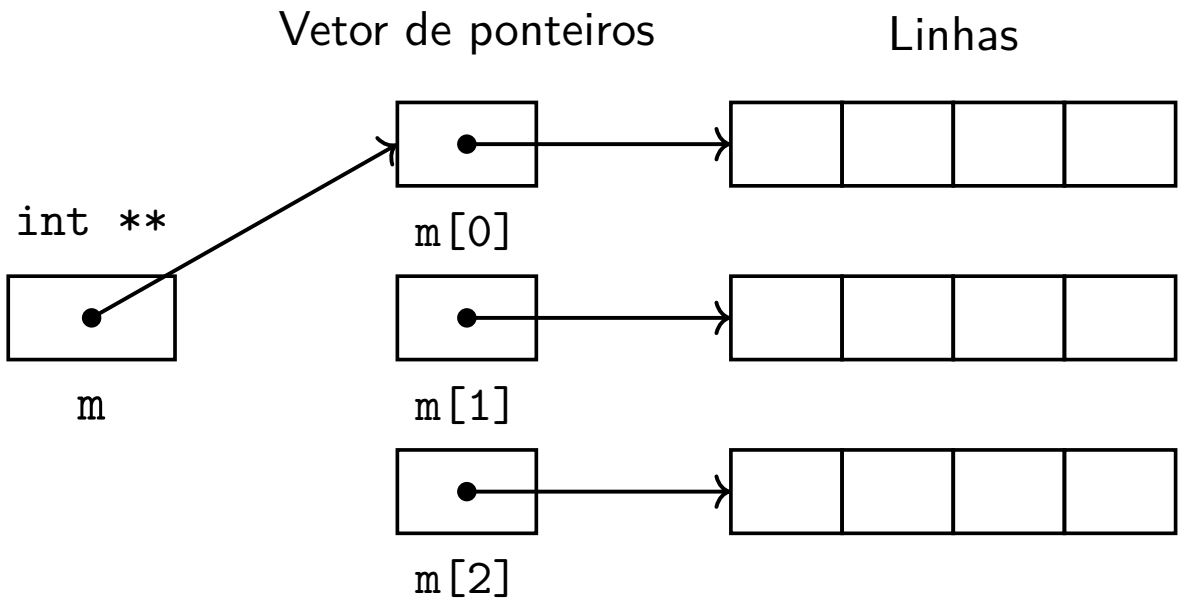
Figura 2: Cada desreferenciação avança um nível: `*pp` é `p` e `**pp` é `x`.



Matriz Dinâmica

- A matriz é montada como um **vetor de ponteiros**, em que cada posição aponta para uma linha.
- Cada linha é um **bloco independente**, e as linhas não ficam necessariamente vizinhas na memória.

Figura 3: Matriz de 3 linhas e 4 colunas construída com um vetor de ponteiros.



Alocando uma Matriz

- Primeiro alocamos o vetor de ponteiros, depois cada uma das linhas:

```
int **m;
m = (int **) malloc(linhas * sizeof(int *));
for (i = 0; i < linhas; i++) {
    m[i] = (int *) malloc(colunas * sizeof(int));
}
```

- São necessárias `linhas + 1` chamadas a `malloc()`.
- O acesso aos elementos é o mesmo de uma matriz de tamanho fixo:

```
m[1][2] = 50;
printf("%d\n", m[1][2]);
```

Liberando uma Matriz

- Cada bloco alocado precisa de uma liberação, começando pelas linhas:

```
for (i = 0; i < linhas; i++) {
    free(m[i]);  // Libera cada linha
}
free(m);         // Libera o vetor de ponteiros
```

- Liberar o vetor de ponteiros primeiro deixaria as linhas inalcançáveis.
- O resultado seria um vazamento de memória de todas as linhas.

Exemplo Completo

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int linhas, colunas;
    int **m;

    printf("Linhas e colunas: ");
    scanf("%d %d", &linhas, &colunas);

    m = (int **) malloc(linhas * sizeof(int *));
    if (m == NULL) {
        return 1;
    }
    for (int i = 0; i < linhas; i++) {
        m[i] = (int *) malloc(colunas * sizeof(int));
    }
```

Exemplo Completo

```
    for (int i = 0; i < linhas; i++) {
        for (int j = 0; j < colunas; j++) {
            scanf("%d", &m[i][j]);
        }
    }
    for (int i = 0; i < linhas; i++) {
        for (int j = 0; j < colunas; j++) {
            printf("%d ", m[i][j]);
        }
        printf("\n");
    }
    for (int i = 0; i < linhas; i++) {
        free(m[i]);
    }
    free(m);
    return 0;
}
```

Estruturas de Dados I

Modularização e Tipos Abstratos de Dados

Professor: Gustavo T. Naozuka
gtnaozuka@uel.br



Universidade
Estadual de Londrina

23 de setembro de 2026

Objetivos da Aula

- Dividir um programa em C em **módulos**.
- **Compilar** projetos com vários arquivos usando o GCC.
- Compreender o conceito de **Tipo Abstrato de Dados (TAD)** e a separação entre interface e implementação.

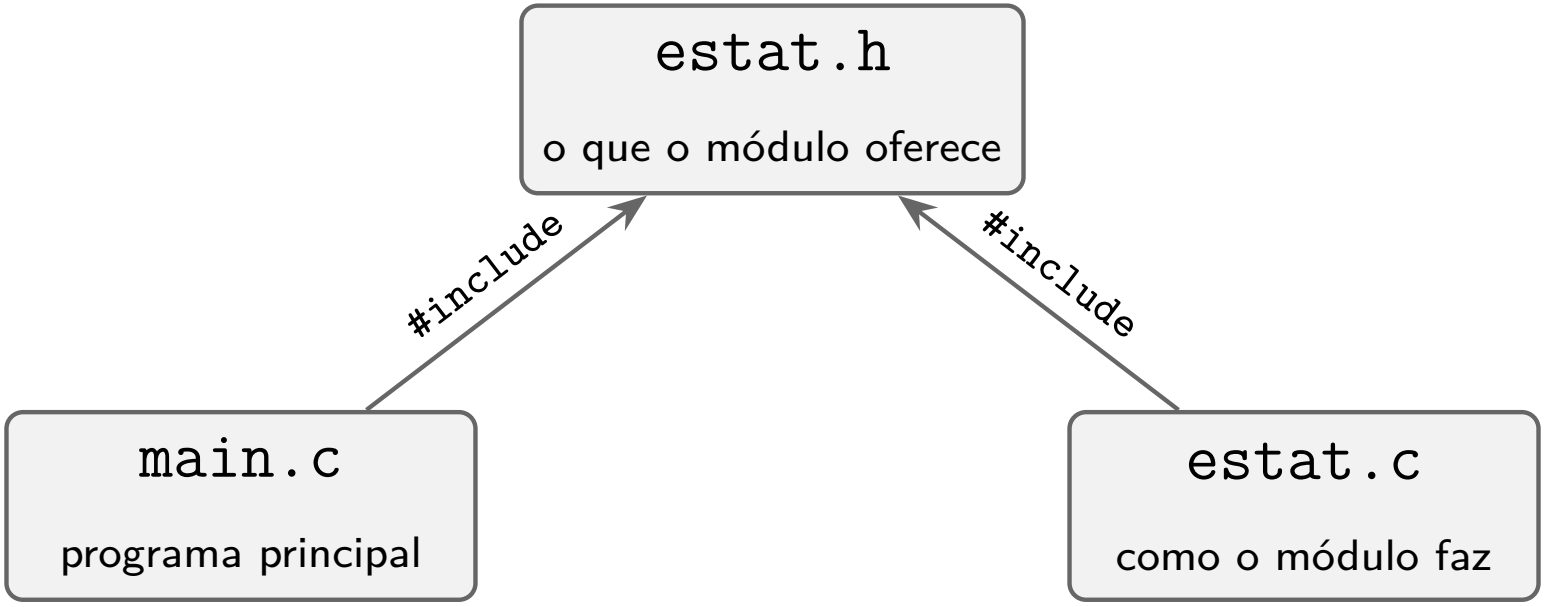
Modularização

Motivação

- Até agora, todos os programas couberam em um **único arquivo** `.c`.
- Isso funciona bem enquanto o programa tem poucas funções.
- À medida que o código cresce, o arquivo único passa a atrapalhar:
  - Centenas ou milhares de linhas para navegar;
  - Funções de assuntos diferentes misturadas;
  - Qualquer alteração exige recompilar tudo.
- A partir de agora, os programas serão divididos em **vários arquivos**.

Modularização

- Um **módulo** agrupa funções de um mesmo assunto.
- Cada módulo é formado por dois arquivos:
  - Arquivo de **cabeçalho** (também chamado de *header*), com extensão `.h`;
  - Arquivo de **implementação**, com extensão `.c`.
- O **programa principal** continua em seu próprio `.c`.



Arquivos de Cabeçalho e de Implementação

- O **arquivo `.h`** contém aquilo que os outros arquivos precisam conhecer:
  - Protótipos das funções oferecidas pelo módulo;
  - Definições de tipos criadas com `typedef`;
  - Constantes definidas com `#define`.
- O **arquivo `.c`** do módulo contém o corpo de cada função declarada no cabeçalho.
- Pode conter também funções auxiliares que só interessam ao próprio módulo.
- O arquivo `.c` **inclui** o seu próprio cabeçalho, para que o compilador verifique se declaração e definição combinam.

Exemplo: Módulo de Estatística

- Considere um módulo com duas funções, média e desvio padrão, aplicadas a um vetor de números reais.
- O programa será dividido em três arquivos:
  - `estat.h`: o que o módulo oferece;
  - `estat.c`: como o módulo faz;
  - `main.c`: o programa que usa o módulo.
- O mesmo módulo poderá ser reaproveitado em qualquer outro programa, sem alteração.

Exemplo: Arquivo `estat.h`

- O **arquivo `estat.h`** contém apenas dois protótipos e algumas diretivas de pré-processador que ainda serão explicadas:

```
#ifndef ESTAT_H
#define ESTAT_H

double media(double v[], int n);

double desvioPadrao(double v[], int n);

#endif
```

Exemplo: Arquivo estat.c

```c
#include <math.h>
#include "estat.h"

double media(double v[], int n) {
    double soma = 0.0;
    for (int i = 0; i < n; i++)
        soma += v[i];
    return soma / n;
}

double desvioPadrao(double v[], int n) {
    double m = media(v, n);
    double soma = 0.0;
    for (int i = 0; i < n; i++)
        soma += (v[i] - m) * (v[i] - m);
    return sqrt(soma / n);
}
```

Exemplo: Arquivo `main.c`

- O **programa principal** usa as duas funções sem saber como foram escritas:

```c
#include <stdio.h>
#include "estat.h"

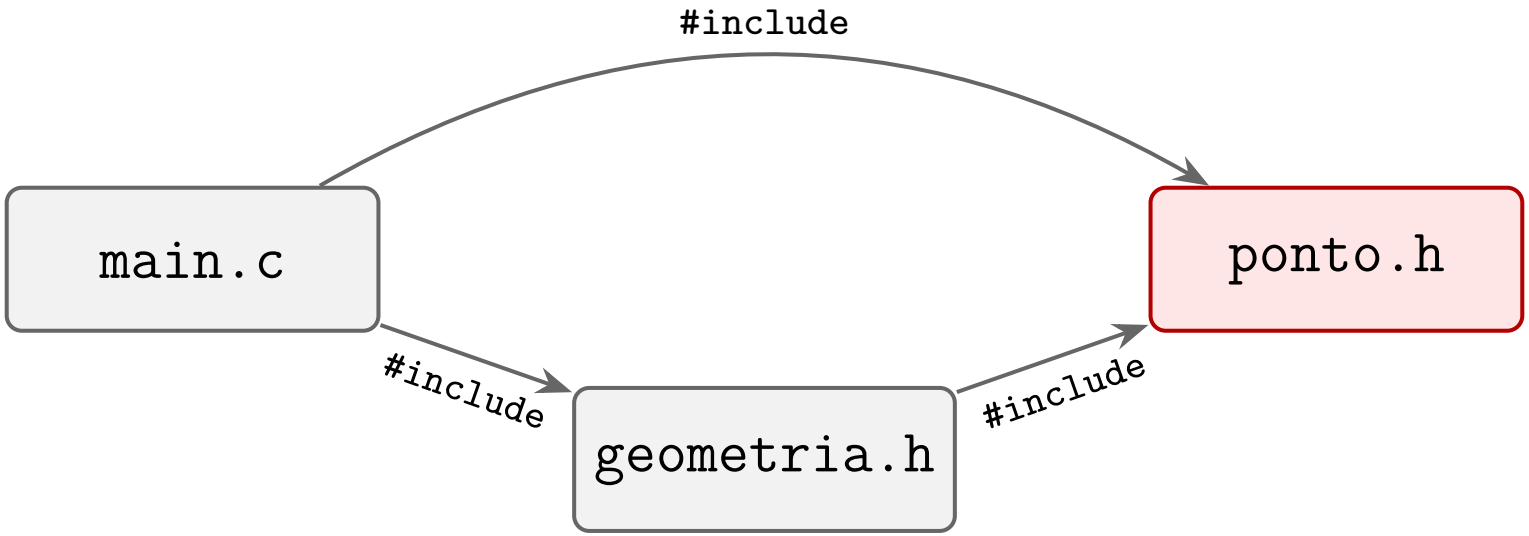
int main(void) {
    double notas[5] = {7.5, 8.0, 6.25, 9.0, 5.5};

    printf("Media: %.2f\n", media(notas, 5));
    printf("Desvio: %.2f\n", desvioPadrao(notas, 5));

    return 0;
}
```

Problema da Inclusão Dupla

- Cabeçalhos podem incluir **outros cabeçalhos**.
- Suponha que o cabeçalho `ponto.h` chega ao programa por dois caminhos: diretamente e através de `geometria.h`.



- O pré-processador copia o conteúdo de `ponto.h` duas vezes.
- O tipo `Ponto` acaba definido duas vezes, e a compilação falha com `redefinition of 'Ponto'`.

Guardas de Inclusão

```
#ifndef ESTAT_H
#define ESTAT_H

...

#endif
```

- `#ifndef ESTAT_H` verifica se o nome `ESTAT_H` foi definido.
- Na primeira inclusão, o nome é **definido**, e o conteúdo é copiado.
- Nas inclusões seguintes, o nome já existe, e todo o trecho é **ignorado**.

Guardas de Inclusão

- O **nome da guarda** costuma ser o nome do arquivo em letras maiúsculas, com o ponto substituído por sublinhado.
- O nome precisa ser **único** no projeto inteiro.
- Todo arquivo `.h` deve ter guarda, enquanto que arquivos `.c` não levam guarda.

Compilação Separada

Processo de Compilação

- Na primeira aula, o comando de compilação pareceu uma etapa única:

```
gcc programa.c -o programa
```

- Na verdade, são três etapas encadeadas:
  - **Pré-processamento**: resolve `#include` e `#define`;
  - **Compilação**: traduz o código C para código de máquina;
  - **Ligação**: junta as partes e produz o executável.
- Com vários arquivos, precisamos **controlar** onde uma etapa termina e a outra começa.

Compilando em Uma Etapa

- A forma mais **direta** é listar todos os arquivos `.c` do projeto:

```
gcc main.c estat.c -o programa -lm
```

- O GCC compila os dois arquivos e liga o resultado em um executável chamado `programa`.
- A opção `-lm` é necessária porque `estat.c` usa `sqrt()`, da biblioteca `math.h`.
- Funciona bem para projetos pequenos, mas recompila **tudo** a cada alteração.

Compilando em Duas Etapas

- A **opção `-c`** interrompe o processo antes da ligação:

```
gcc -c estat.c      # gera estat.o
gcc -c main.c       # gera main.o
gcc main.o estat.o -o programa -lm
```

- Cada `.c` gera um arquivo-objeto com extensão `.o`.
- O último comando faz apenas a **ligação**, sem recompilar nada.
- Nesse caso, precisamos recompilar **apenas** os arquivos `.c` alterados.
- Se o arquivo `.h` mudar, **todos** os `.c` que o incluem precisam ser recompilados.

Arquivo-Objeto

- O **arquivo-objeto** é um arquivo com código de máquina já traduzido, mas ainda incompleto.
- No exemplo, o arquivo `estat.o` contém o código de máquina das funções `media()` e `desvioPadrao()`.
- O arquivo `main.o` contém o código da função `main()`, com lacunas: ele chama as funções, mas não sabe onde elas estão.
- O ligador preenche as lacunas, associando cada chamada à função correspondente em outro arquivo-objeto.

Tipos Abstratos de Dados

Tipo Abstrato de Dados (TAD)

- Um **TAD** é definido por dois componentes:
  - Conjunto de valores que ele pode assumir;
  - Conjunto de operações que podem ser feitas sobre esses valores.
- Por exemplo, um TAD Conjunto de inteiros:
  - Valores: coleções de inteiros, sem repetição e sem ordem definida;
  - Operações: inserir, remover, verificar se um elemento pertence e informar a cardinalidade.
- Quem usa o TAD conhece apenas as **operações**: a forma de armazenamento permanece oculta.
- O adjetivo **abstrato** se refere exatamente a essa omissão.

Tipo Concreto × Tipo Abstrato

- Um **tipo concreto** é aquele cuja representação interna faz parte do que o usuário conhece: ele lê os campos e pode alterá-los livremente.
- **Exemplo:** Um registro `Aluno`, criado com `struct` e `typedef`, é um tipo concreto.

```
typedef struct {
    char nome[50];
    int matricula;
    float media;
} Aluno;

Aluno a;
a.media = 9.5;
```

- Em um **tipo abstrato**, o usuário recebe apenas as operações: a representação interna permanece oculta.

Interface e Implementação

- A **interface** é o conjunto de operações oferecidas, com seus parâmetros e retornos.
- A **implementação** é o código que realiza essas operações e a estrutura de dados escolhida para isso.
- A separação entre interface e implementação é feita pela **modularização**:
  - O arquivo `.h` é a interface;
  - O arquivo `.c` é a implementação.

Interface como Contrato

- A interface deve deixar claro, para cada operação:
  - O que ela exige antes de ser chamada;
  - O que ela garante ao terminar;
  - O que ela devolve quando a operação não é possível.
- Em C, essas informações ficam em **comentários** no próprio arquivo de cabeçalho.

Problema do Registro no Cabeçalho

- Considere o seguinte trecho do arquivo `contador.h`:

```
typedef struct {
    int valor;
} Contador;

void zerar(Contador *c);
void incrementar(Contador *c);
int lerValor(Contador *c);
```

- As operações estão definidas no `contador.c`, e o módulo funciona.
- No entanto, o registro está **visível** para quem incluir o cabeçalho.

Problema do Registro no Cabeçalho

- Nada impede o acesso direto ao campo `valor`:

```
Contador c;
zerar(&c);
incrementar(&c);

c.valor = 1000; // Campo alterado diretamente
c.valor = -7; // Campo com valor inválido
```

- As operações do módulo deixam de ser o único caminho para modificar o contador.

Solução: Ponteiro Opaco

- O cabeçalho anuncia que existe um registro chamado `contador`, mas não diz quais são seus campos:

```
typedef struct contador Contador;
```

- Isso é chamado de **declaração incompleta de tipo**.
- O compilador **aceita** ponteiros para `Contador`, porque todo ponteiro ocupa o mesmo espaço.
- Ele **não aceita** declarar uma variável do tipo `Contador`.

Solução: Arquivo contador.h

```
#ifndef CONTADOR_H
#define CONTADOR_H

typedef struct contador Contador;

Contador *criarContador(void);
void destruirContador(Contador *c);

void incrementar(Contador *c);
int lerValor(Contador *c);

#endif
```

- Como o usuário não pode declarar a variável, o módulo precisa criá-la e liberá-la.

Solução: Arquivo contador.c

```c
#include <stdlib.h>
#include "contador.h"

struct contador {
    int valor;
};

Contador *criarContador(void) {
    Contador *c = (Contador *) malloc(sizeof(Contador));
    if (c != NULL)
        c->valor = 0;
    return c;
}

void destruirContador(Contador *c) { free(c); }
void incrementar(Contador *c) { c->valor++; }
int lerValor(Contador *c) { return c->valor; }
```

Solução: Arquivo `main.c`

```c
#include <stdio.h>
#include "contador.h"

int main(void) {
    Contador *c = criarContador();
    if (c == NULL) return 1;

    incrementar(c);
    incrementar(c);
    printf("Valor: %d\n", lerValor(c));

    /* c->valor = 1000; // Erro de compilação */

    destruirContador(c);
    return 0;
}
```

Consequências de Utilizar o Ponteiro Opaco

- O TAD é sempre manipulado por **ponteiro**.
- Toda operação recebe esse ponteiro como **primeiro parâmetro**.
- A **criação** e a **destruição** passam a ser operações obrigatórias da interface.
- A representação interna fica **protegida**.

Estruturas de Dados I

TAD Pilha

Professor: Gustavo T. Naozuka
gtnaozuka@uel.br



Universidade
Estadual de Londrina

24 de setembro de 2026

Objetivos da Aula

- Definir o **TAD Pilha** e sua interface.
- Desenvolver o TAD Pilha por meio de uma **implementação sequencial**.
- Desenvolver o TAD Pilha por meio de uma **implementação encadeada**.

Introdução

Motivação

- Em muitas situações, a ordem em que os elementos saem é determinada pela ordem em que entraram.
- Considere uma **pilha** de pratos sobre a bancada:
  - Cada prato lavado é colocado sobre os anteriores;
  - O prato retirado para uso é o de cima.
- O último prato a entrar é o primeiro a sair.
- Nenhum prato do meio pode ser retirado sem antes remover os que estão acima dele.

Regra LIFO

- Estruturas com esse comportamento são chamadas de **LIFO***.
- **Consequências:**
  - Só existe um ponto de acesso, chamado de **topo**;
  - O elemento mais antigo é o menos acessível.
- Não há busca, não há acesso por posição e não há remoção pelo meio.
- Essa limitação torna as operações simples e rápidas.

*Last In, First Out

Aplicações de Pilhas

- **Desfazer:** o editor de texto guarda cada ação executada; o comando desfazer retira a mais recente.
- **Histórico do navegador:** o botão voltar retorna à última página visitada.
- **Chamadas de função:** cada função chamada é empilhada e só é retirada quando termina.
- **Expressões aritméticas:** os operadores que ainda não podem ser aplicados ficam guardados até que seus operandos estejam determinados.

Interface

Operações do TAD Pilha

- **Criar:** produzir uma pilha vazia.
- **Destruir:** liberar a pilha e tudo o que ela contém.
- **Empilhar:** inserir um elemento no topo.
- **Desempilhar:** remover o elemento do topo e devolvê-lo.
- **Consultar o topo:** obter o elemento do topo sem removê-lo.
- **Verificar se está vazia.**
- **Informar o tamanho.**

Decisões de Contrato e Tipo dos Elementos

- A criação devolve `NULL` se não houver memória disponível.
- Operações impossíveis, como desempilhar de uma pilha vazia, devolvem `false` e não alteram a pilha.
- O elemento removido é copiado para um endereço fornecido pelo usuário.
- O tipo dos elementos armazenados recebe um nome próprio:

```
typedef int Item;
```

- Para guardar números reais ou registros completos, basta alterar essa única linha.

Arquivo pilha.h

```c
#ifndef PILHA_H
#define PILHA_H

#include <stdbool.h>

typedef int Item;
typedef struct pilha Pilha;

Pilha *criarPilha(void);
void destruirPilha(Pilha *p);

bool empilhar(Pilha *p, Item x);
bool desempilhar(Pilha *p, Item *x);
bool consultarTopo(Pilha *p, Item *x);

bool pilhaVazia(Pilha *p);
int tamanhoPilha(Pilha *p);

#endif
```

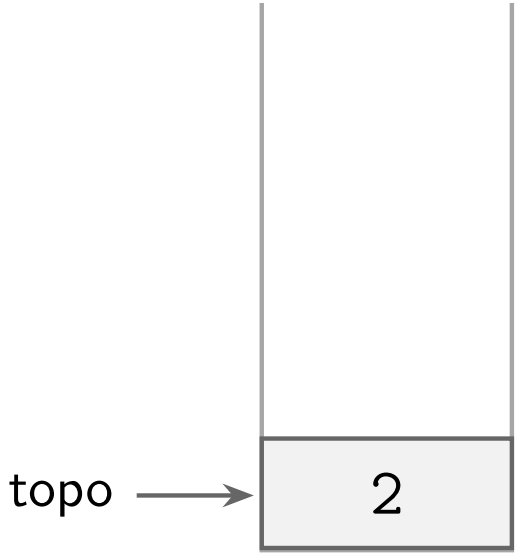
Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Operação: `Pilha *p = criarPilha();`



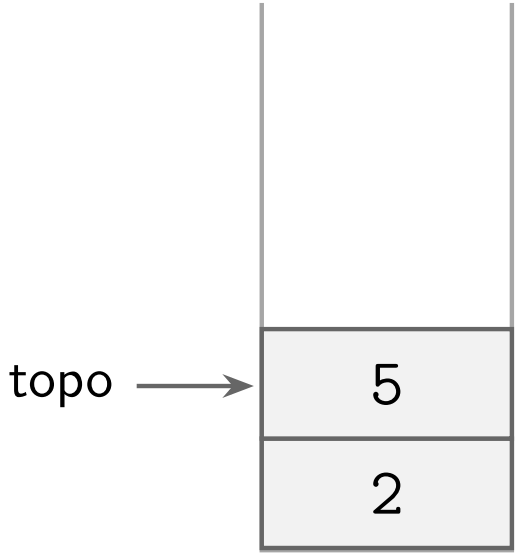
Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Lido: `2` ⇒ Operação: `empilhar(p, 2)`



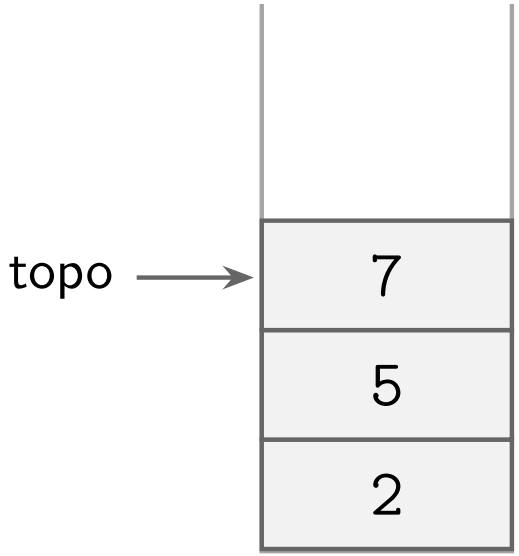
Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Lido: `5` ⇒ Operação: `empilhar(p, 5)`



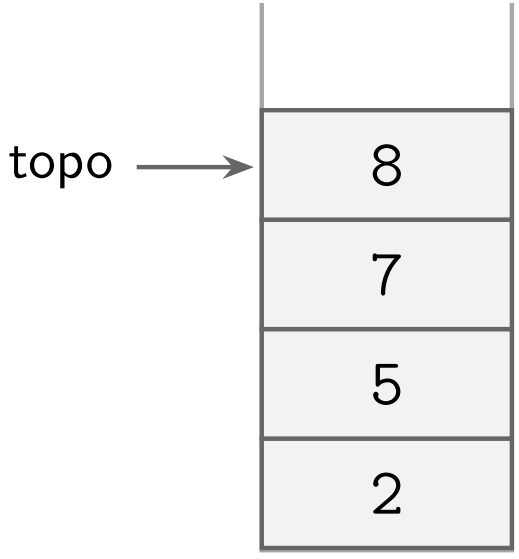
Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Lido: `7` ⇒ Operação: `empilhar(p, 7)`



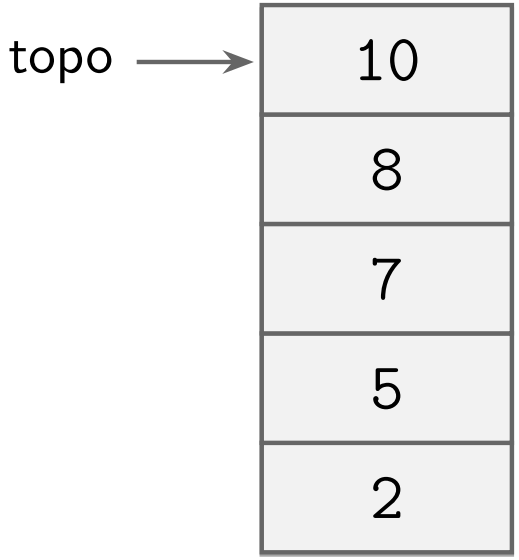
Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Lido: `8` ⇒ Operação: `empilhar(p, 8)`



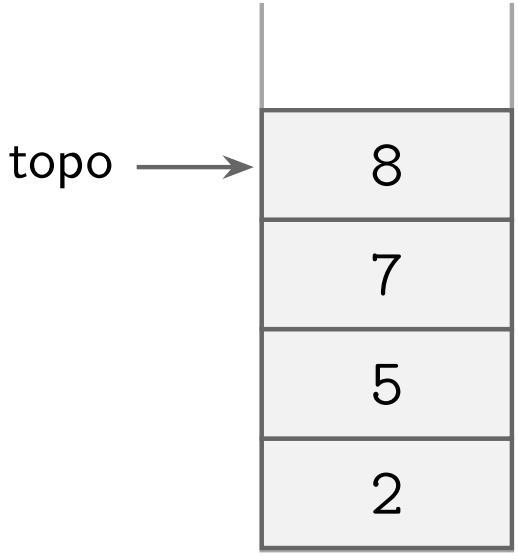
Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Lido: `10` ⇒ Operação: `empilhar(p, 10)`



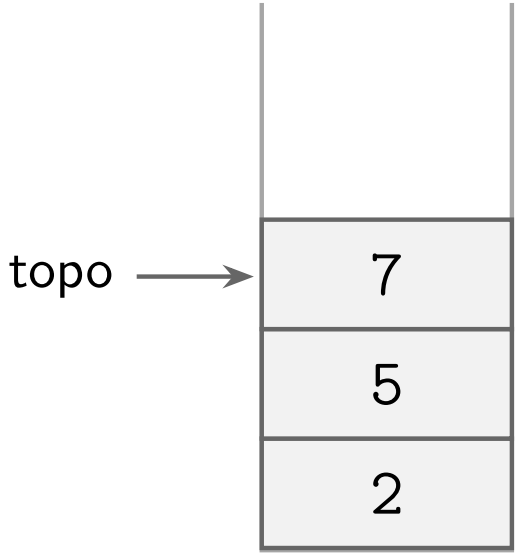
Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Operação: `desempilhar(p, &item)` ⇒ Saída: `10`



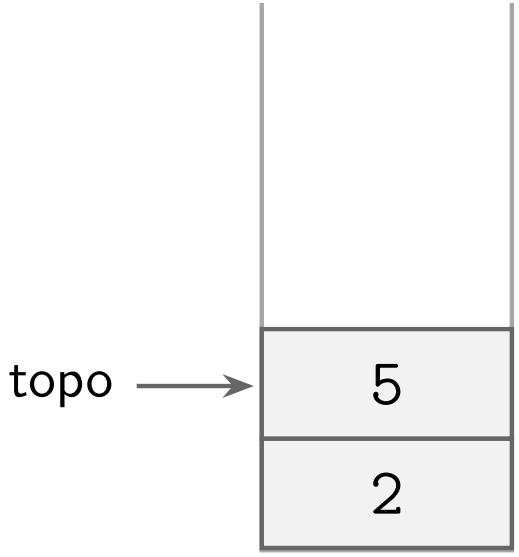
Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Operação: `desempilhar(p, &item)` ⇒ Saída: `10 8`



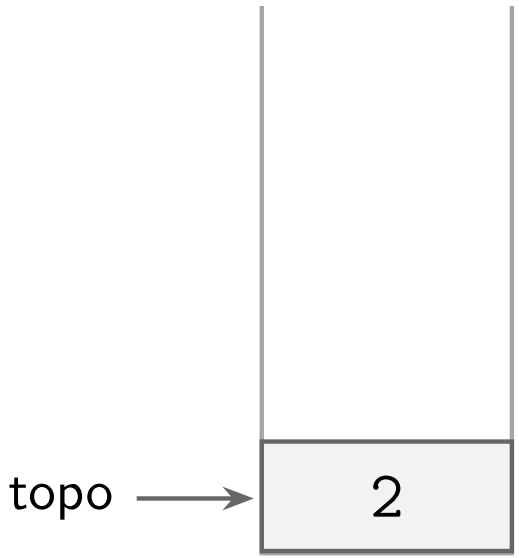
Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Operação: `desempilhar(p, &item)` ⇒ Saída: `10 8 7`



Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Operação: `desempilhar(p, &item)` ⇒ Saída: `10 8 7 5`



Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Operação: `desempilhar(p, &item)` ⇒ Saída: `10 8 7 5 2`



Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Operação: `destruirPilha(p)` $\Rightarrow$ A pilha é liberada.

Exemplo: Inversão de uma Sequência

```c
#include <stdio.h>
#include "pilha.h"

int main() {
    int entrada[5] = {2, 5, 7, 8, 10};
    Item item;

    Pilha *p = criarPilha();
    if (p == NULL)
        return 1;

    for (int i = 0; i < 5; i++)
        empilhar(p, entrada[i]);
    ...
```

Exemplo: Inversão de uma Sequência

```
    ...
    while (desempilhar(p, &item))
        printf("%d ", item);

    destruirPilha(p);
    return 0;
}
```

Implementação Sequencial

Implementação Sequencial

- Na implementação sequencial, os elementos são guardados em um vetor de capacidade **fixa**.
- Um campo inteiro registra a **posição** do elemento que está no topo.
- Empilhar **avança** essa posição e escreve no vetor; desempilhar **recua**.

Arquivo pilha_seq.c

Registro

```c
#include <stdlib.h>
#include "pilha.h"

#define MAX 100

struct pilha {
    Item itens[MAX];
    int topo;
};
```

Representando a Pilha Vazia

- O **campo** `topo` indica a posição do último elemento inserido no vetor.
- Com um elemento, `topo = 0`; com dois elementos, `topo = 1`...
- Para a pilha vazia, a **convenção** é `topo = -1`.
- O **número de elementos** é sempre `topo + 1`, inclusive quando a pilha está vazia.

Arquivo pilha_seq.c

Criação, Destruição e Consultas Simples

```
Pilha *criarPilha(void) {
    Pilha *p = (Pilha *) malloc(sizeof(Pilha));
    if (p != NULL)
        p->topo = -1;
    return p;
}

void destruirPilha(Pilha *p) { free(p); }

bool pilhaVazia(Pilha *p) { return p->topo == -1; }

int tamanhoPilha(Pilha *p) { return p->topo + 1; }
```

Arquivo `pilha_seq.c`

Inserção

```
bool empilhar(Pilha *p, Item x) {
    if (p->topo == MAX - 1)
        return false;

    p->topo++;
    p->itens[p->topo] = x;
    return true;
}
```

- O teste inicial verifica se o vetor já está **cheio**.

Arquivo `pilha_seq.c`

Remoção

```
bool desempilhar(Pilha *p, Item *x) {
    if (p->topo == -1)
        return false;

    *x = p->itens[p->topo];
    p->topo--;
    return true;
}
```

- O elemento é **copiado** para o endereço fornecido pelo usuário antes de o topo recuar.
- O valor permanece fisicamente no vetor, mas passa a estar **fora** da região válida.

Arquivo `pilha_seq.c`

Consulta ao Topo

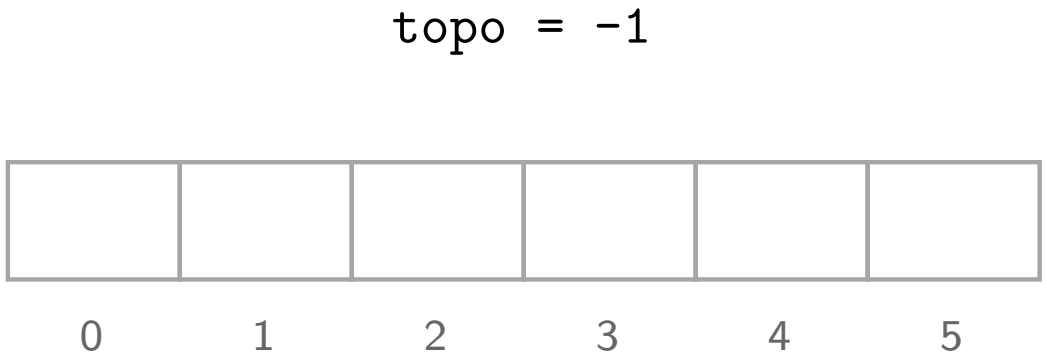
```
bool consultarTopo(Pilha *p, Item *x) {
    if (p->topo == -1)
        return false;

    *x = p->itens[p->topo];
    return true;
}
```

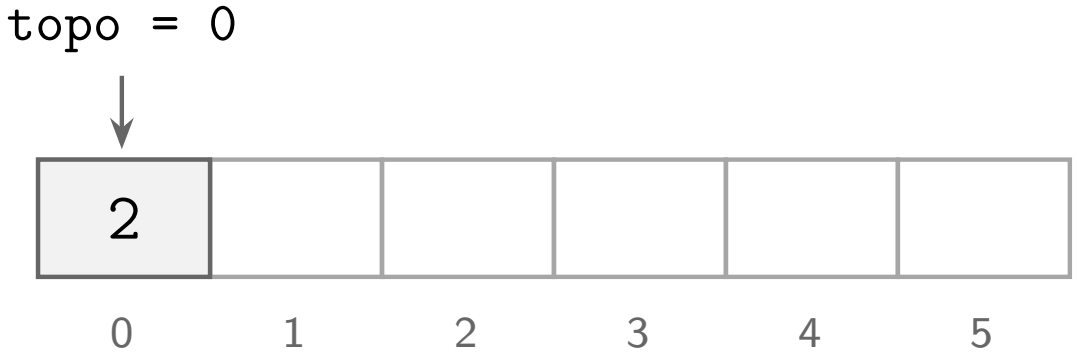
- Essa operação é útil quando o programa precisa decidir algo com base no topo antes de removê-lo.

Exemplo: Implementação Sequencial

1. `criarPilha()`

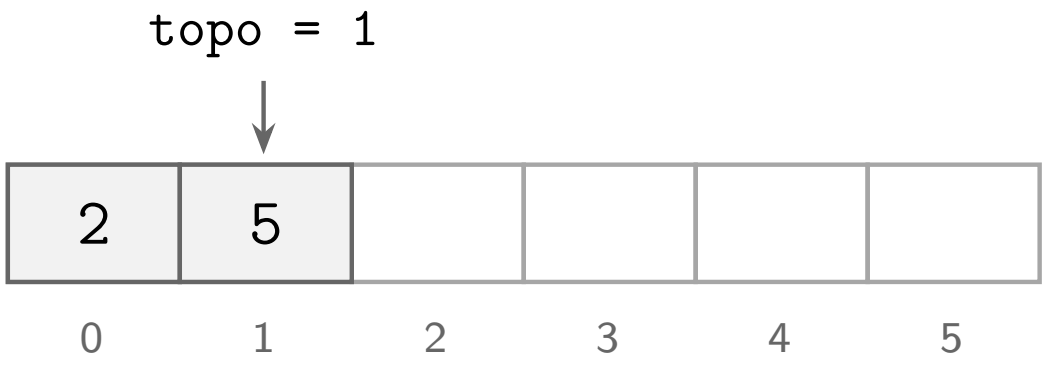


2. `empilhar(p, 2)`

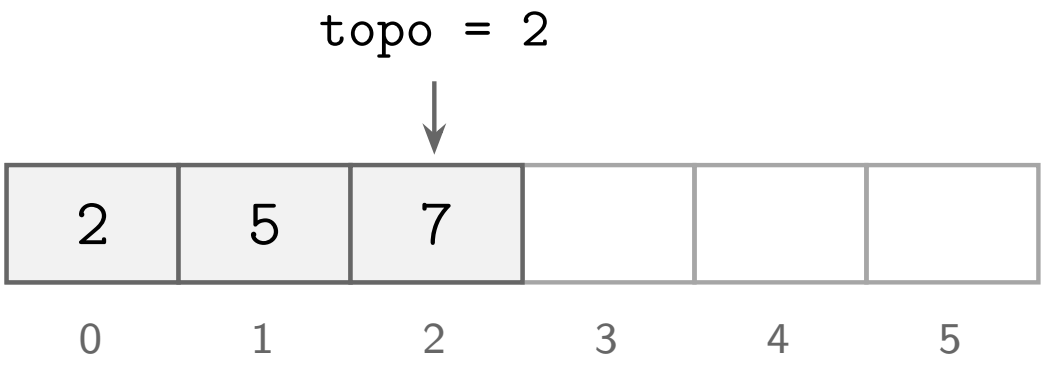


Exemplo: Implementação Sequencial

3. `empilhar(p, 5)`

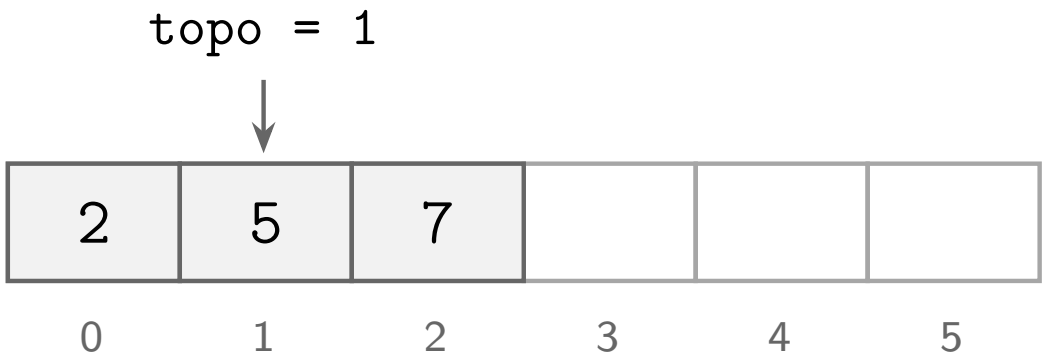


4. `empilhar(p, 7)`

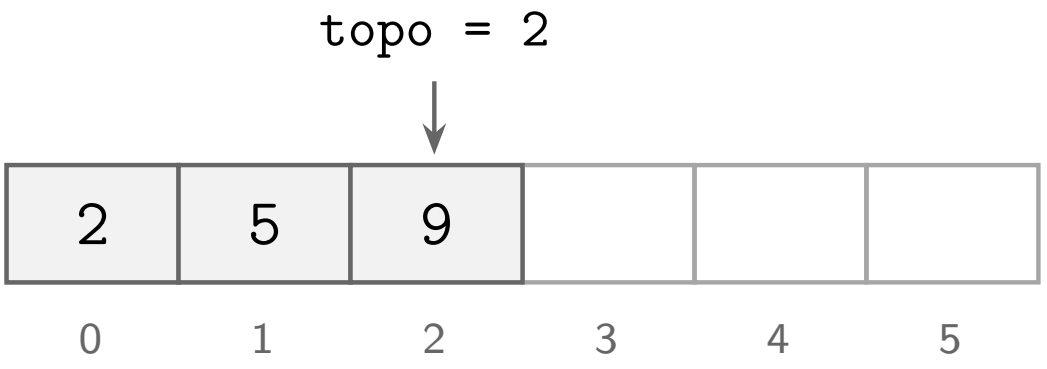


Exemplo: Implementação Sequencial

5. `desempilhar(p, &item)` devolve 7, e o topo recua.



6. `empilhar(p, 9)`



Custo das Operações

- Todas as operações executam um número fixo de comandos.
- Nenhuma delas contém laço, e nenhuma depende da quantidade de elementos armazenados.
- Dizemos que essas operações têm **custo constante**.

Limitações

- **Capacidade fixa:** a pilha não comporta mais do que `MAX` elementos.
- **Memória ociosa:** o vetor inteiro é reservado na criação, antes de saber quantos elementos serão empilhados.
- A alternativa é reservar memória a cada inserção, o que leva à **implementação encadeada**.